

PROGRAM 3

Code a dockerized python flask application

Step 1 : Create the following files inside a folder (DockerFile , app.py, requirements.txt)

```
GNU nano 7.2
FROM python:latest

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

```
GNU nano 7.2
from flask import Flask;
app = Flask(__name__);
@app.route("/")
def hello():
    return "Hello, World!";
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000);
```

```
GNU nano 7.2
Flask
```

Step 2 : Open that in a terminal. Run these commands to build and run Docker

```
lrv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/p3$ nano Dockerfile
lrv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/p3$ nano app.py
lrv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/p3$ nano requirements.txt
lrv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/p3$ docker build -t p3 .
[+] Building 7.3s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 181B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:latest@sha256:1ad1a43b5e2478e62056bbc28028afd858185d73bf4d6a439cbb058b6800a96d
=> [internal] load build context
=> => transferring context: 441B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:1f90bf7c1a58c846dd4f7836664c4b1ea197d457ed78728f9436a038c9502e31
=> => naming to docker.io/library/p3
lrv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/p3$ docker run -t -p 5000:5000 p3
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
Press CTRL+C to quit
```

Step 3 : The output of the program

